

CROSS: Sistema di Exchange Order Book

Relazione sintetica - Laboratorio di Reti

Ernesto Cioli mat. 639496

2 luglio 2025

1 Scelte progettuali e interpretazioni

- **Interruzione server:** E' gestito il caso in cui il server si disconnette, mandando una `Response` al client in modo asincrono, gestita da un receiver thread di `TcpClient`.
- **Interruzione client:** E' gestito il caso in cui il client si disconnette, mandando una speciale richiesta di `logout()` al server, chiudendo il socket su entrambi i lati.
- **Threadpool socket:** Dato il numero incognito di utenti contemporaneamente interessati, è stato scelto un `CachedThreadPool`, perché più flessibile di un `FixedThreadPool`,
- **Durata degli stop orders:** Sono gestiti come *day orders*, ovvero alla fine della sessione (chiusura del server) vengono eliminati.
- **Timeout con stop order:** E' gestita l'interruzione del timeout per il client socket, se l'utente ha uno stop order al momento attivo (vedi thread `keepAlive`).
- **Formato Request:** `Request` deve essere molto malleabile ed è stato scelto un approccio più meccanico che non di creazione di sottoclassi: i `Request.values` assumono la forma di una `Map<String, Object>`, i cui oggetti possono essere stringhe o interi, in base alla necessità (la serializzazione è gestita grazie a `ConditionalMapTypeAdapter`).
- **Response, Notification, Trade, User** → assumono le proprietà delle rappresentazioni JSON richieste, con eventuali sottoclassi e sottotipi.
- **Formato Order** → le sottoclassi di `Order` sono deserializzate con `OrderTypeAdapter` a causa delle diverse sottoclassi che possiede, variabili in base al campo `orderType`.
- **Storico ordini** → Per la sua grande quantità di dati, è stato deciso di prendere il file fornito dalla docente `storicoOrdini.json` come punto di partenza del file degli ordini `trades_file.json` (se vuoto). Saranno adeguati a tale modifica anche il generatore di `orderId` e i calcoli richiesti da `getPriceHistory()`.

2 Schema dei Thread

2.1 Lato Server

- **ServerMain:** Gestisce l'avvio del server e la lettura della configurazione. Prevede la creazione e gestione di: server TCP, notificatore UDP, orderbook, userbook.
- **ShutdownHook:** Gestisce la corretta chiusura di thread, socket e altre strutture dati del server al momento della terminazione.
- **TcpWorker (pool):** `CachedThreadPool` di gestione delle richieste client. Il server tiene comunque traccia dei singoli thread attivi per una corretta gestione delle interruzioni.

- **UdpNotifier (async, daemon):** Invia una notifica UDP al client dopo una breve attesa. Necessario per non bloccare il client sull'attesa della notifica e permettere un'eventuale ricezione tempestiva della Response su TCP.
- **PersistenceManager (scheduler):** Thread programmato a ripetizione periodica per il salvataggio delle strutture dati su file.

2.2 Lato Client

- **ClientMain:** Gestisce l'interfaccia CLI, l'input utente, la lettura della configurazione. Prevede la creazione e gestione di: client TCP, ricevitore UDP, formattazione Request.
- **ShutdownHook:** Gestisce la corretta chiusura di thread, socket e altre strutture dati del client al momento della terminazione.
- **TcpReceiver (async):** Dal TcpClient è attivato per la ricezione delle Response dal server in modo asincrono e non bloccante. Oltre al normale compito, svolge un ruolo fondamentale per la ricezione del segnale di interruzione del server, che indica la sua terminazione e, conseguentemente, di dover terminare il client.
- **keepAlive (async, daemon):** Sempre dal TcpClient è attivato per l'invio periodico di segnali al server, per resettare il timeout impostato sul socket TCP dal server.
- **UdpListener (async):** Si mette in ascolto delle notifiche UDP provenienti dal server: ordini evasi o stop order convertiti e poi falliti.

3 Strutture dati implementate

- **ConcurrentHashMap** (orderMap, tradeMap, userMap) → rispetto a una normale HashMap, riesce a garantire operazioni thread-safe, senza corruzione o sovrapposizione (sia il server che i tcpWorker compiono operazioni sulla mappa)
- **ConcurrentSkipListSet** (bidBook, askBook) → oltre a garantire operazioni thread-safe, è un albero binario rosso-nero con ordinamento automatico su più attributi. E' la struttura che permette di mantenere un order matching secondo una price-time priority
- **LinkedHashMap** (clientPorts) → rispetto a una HashMap, mantiene l'ordine di inserimento per i suoi elementi
- **LinkedList, ArrayList** (stopOrders, trades, ...) → usate in modo sparso, nonostante principali differenze nelle velocità di operazioni CRUD, per le piccole dimensioni delle situazioni in cui sono state usate, la differenza è quasi trascurabile
- **AtomicInteger, volatile** (idGenerator, running, udpStarted) → utilizzati per l'atomicità delle operazioni, rendendo più thread-safe i contesti di utilizzo

4 Primitive di sincronizzazione

- **metodi synchronized** (insertOrder, cancelOrder, ...): Per consistenza e come ulteriore protezione in un ambiente acceduto da vari thread, alcuni metodi sono sincronizzati, affinché non ci siano sovrapposizioni di utilizzo in una stessa istanza di classe.
- **ConcurrentHashMap, ConcurrentSkipListSet, AtomicInteger, volatile:** Le strutture già descritte partecipano ad un ambiente concorrente più sicuro.

4.1 Meccanismi di coordinamento:

- **Timeout socket:** Timeout regolabile sui client TCP socket
- **Shutdown controllato:** Chiusura ordinata di thread e socket grazie agli `ShutdownHook`
- **Gestione eccezioni:** Speciale gestione delle eccezioni, soprattutto per gli errori di rete o di I/O

5 Istruzioni di compilazione ed esecuzione

5.1 Prerequisiti

- Java 11 o superiore (alcuni metodi sono compatibili solo con Java 11+)
- Terminale: **Command prompt** di Windows (non Powershell) oppure **s7 Ubuntu/wsl**
- Librerie esterne:
 - Gson (per JSON format, in `lib/gson-2.13.1.jar`)
 - Jansi (per AnsiConsole, in `lib/jansi-2.4.2.jar`)
- **Trovarsi nella home directory del progetto:** `/cross/`
(cercare le dir `./src/` e `./lib/` per sapere di essere nella directory corretta)

5.2 Compilazione

- Con javac: `javac -d target/classes -cp "lib/*" @sources.txt`
- Utilizzo diretto dei file JAR **precompilati**

5.3 Esecuzione

- Dopo javac, con le classi compilate in `target/classes` (in Linux, sostituire ogni `';` con `':`):

– **Server:** `java -cp target/classes;lib/* it.unipi.cross.server.ServerMain`

– **Client:** `java --enable-native-access=ALL-UNNAMED
-cp target/classes;lib/* it.unipi.cross.client.ClientMain`

- Con i file JAR precompilati:

– **Server:** `java -jar ./cross-server.jar`

– **Client:** `java --enable-native-access=ALL-UNNAMED -jar ./cross-client.jar`

5.4 Comandi client

Una volta connesso, il client supporta i seguenti comandi:

- `register(username, password)` - Registrazione nuovo utente
- `login(username, password)` - Login utente
- `logout()` - Logout e chiusura
- `updateCredentials(username, oldPwd, newPwd)` - Cambio password
- `insertLimitOrder(type, size, limitPrice)` - Inserimento limit order
- `insertMarketOrder(type, size)` - Inserimento market order
- `insertStopOrder(type, size, stopPrice)` - Inserimento stop order
- `cancelOrder(orderID)` - Cancellazione ordine
- `getPriceHistory(month)` - Storico prezzi per mese
- `help()` - Visualizza aiuto comandi
- `exit()` - Chiusura

Dove:

- `type`: "bid" o "ask"
- `size, price, stopPrice`: numeri interi positivi
- `month`: month in formato "MMYYYY"

5.5 File di configurazione

Il sistema utilizza i seguenti file per i parametri in `./src/main/resources/`:

- `server_config.properties`: Configurazioni server (tcp, udp, persistenza)
- `client_config.properties`: Configurazioni client (tcp, udp)
- `sources.txt`: Elenco file sorgenti da compilare con `javac`

Il sistema utilizza i seguenti file per la persistenza in `./src/main/resources/` (se non esistono, li crea):

- `storedOrders.json`: Limit orders attivi
- `storedUsers.json`: Database utenti (username e password)
- `storedTrades.json`: Database ordini evasi (comprende anche `storicoOrdini.json`)

5.6 Dati forniti per il testing

- `storedUsers.json`: in formato `{username, password}`

```
{giulia, password}
```

```
{matteo, ciao}
```

```
{stefano, 12345}
```

- `storedOrders.json`: in formato `Order` (con timestamp)

Lato ask:

```
{orderId: 1, username; "giulia", size: 25, price: 1110}
```

```
{orderId: 3, username; "stefano", size: 60, price: 1109}
```

Lato bid:

```
{orderId: 2, username; "giulia", size: 25, price: 1105}
```

```
{orderId: 4, username; "matteo", size: 100, price: 1102}
```

- `storedTrades.json`: Viene inizializzato con `storicoOrdini.json` all'avvio